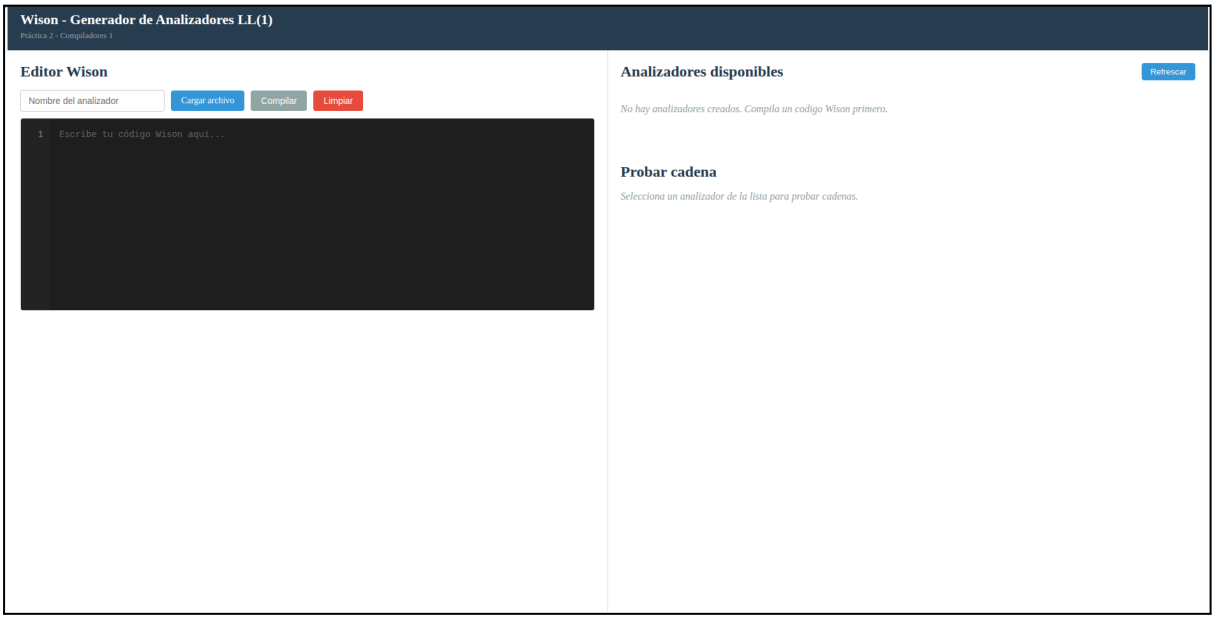
Manual De Usuario
Generador de Analizadores

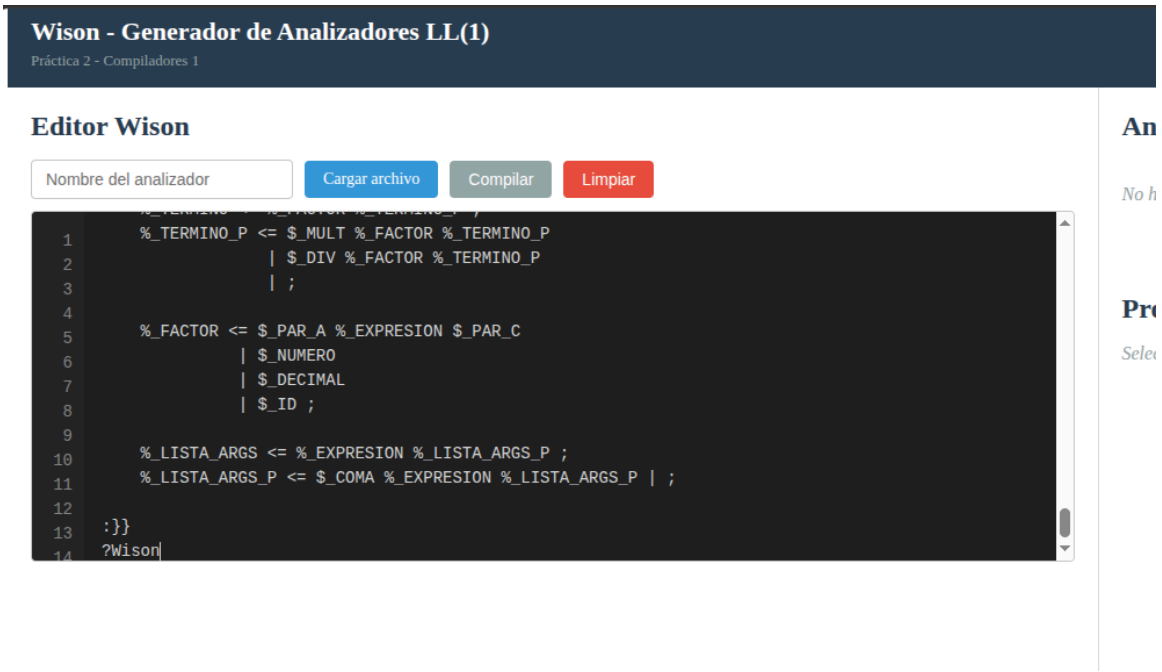
Pantalla principal:

la aplicacion se diseño para que el usuario tenga la comodidad de tener todo a la mano durante en uso de esta
en la pantalla principal estan todas las herramientas que se necesitan el uso correcto de la aplicacion

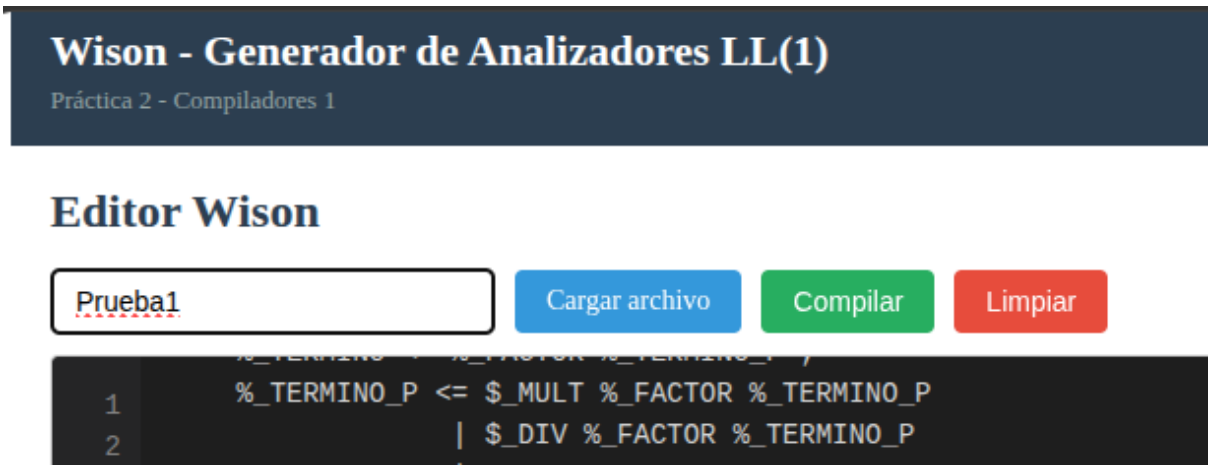


1. Editor de codigo:

esta parte es en donde el usuario podar ya sea ingresar codigo o cargar un archivo con el codigo siempre y cuando este tenga el lenguaje wison ya que de lo contrario se tratara de analizar pero lanzara errores



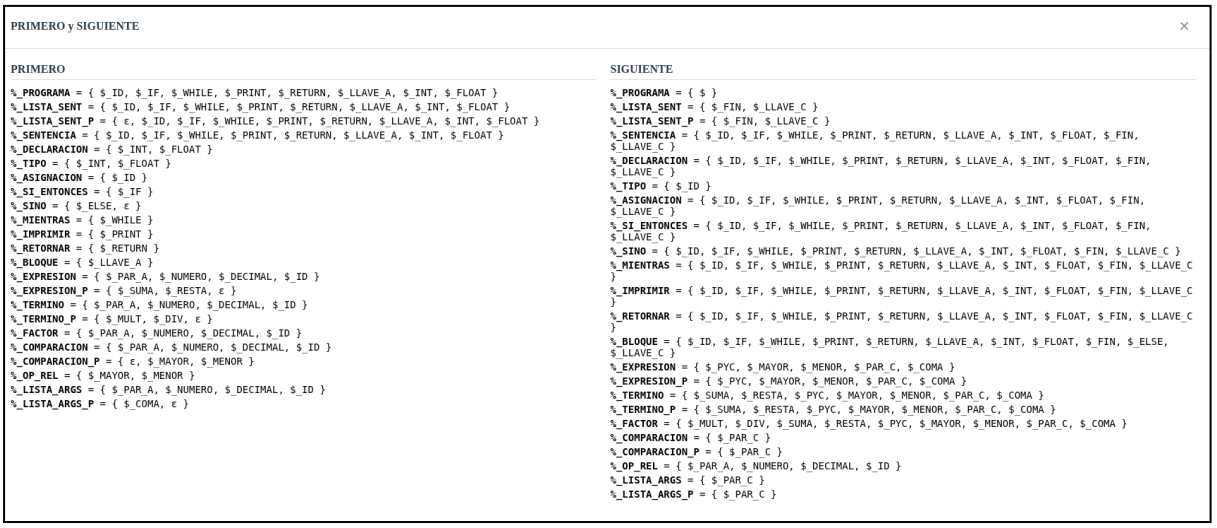
una vez el usuario haya escrito o cargado el codigo el sistema requiere que se le asigne un nombre para poder compilarlo de lo contrario no se podra, esto es eficienta para poder identificar a los analizadores que existirar y poder elegir alguno identificandolo correctamente



una vez ingresado el nombre del analizador, si este es correcto entonces se compilara y se mostrara en analizador ya creado en la parte derecha de la pantalla en donde el usuario podra seleccionarlo para poner a prueba algunas cadenas de entrada

2. Elementos del analizador

cuando un analizador se crea correctamente entonces se mostrara tanto los primeros como los siguientes



PRIMERO y SIGUIENTE

PRIMERO

%_PROGRAMA = { $_ID, $_IF, $_WHILE, $_PRINT, $_RETURN, $_LLAVE_A, $_INT, $_FLOAT }
%_LISTA_SENT = { $_ID, $_IF, $_WHILE, $_PRINT, $_RETURN, $_LLAVE_A, $_INT, $_FLOAT }
%_LISTA_SENT_P = { ε, $_ID, $_IF, $_WHILE, $_PRINT, $_RETURN, $_LLAVE_A, $_INT, $_FLOAT }
%_SENTENCIA = { $_ID, $_IF, $_WHILE, $_PRINT, $_RETURN, $_LLAVE_A, $_INT, $_FLOAT }
%_DECLARACION = { $_INT, $_FLOAT }
%_TIPO = { $_INT, $_FLOAT }
%_ASIGNACION = { $_ID }
%_SI_ENTONCES = { $_IF }
%_SINO = { $_ELSE, ε }
%_MIENTRAS = { $_WHILE }
%_IMPRIMIR = { $_PRINT }
%_RETORNAR = { $_RETURN }
%_BLOQUE = { $_LLAVE_A }
%_EXPRESION = { $_PAR_A, $_NUMERO, $_DECIMAL, $_ID }
%_EXPRESION_P = { $_SUMA, $_RESTA, ε }
%_TERMINO = { $_PAR_A, $_NUMERO, $_DECIMAL, $_ID }
%_TERMINO_P = { $_MULT, $_DIV, ε }
%_FACTOR = { $_PAR_A, $_NUMERO, $_DECIMAL, $_ID }
%_COMPARACION = { $_PAR_A, $_NUMERO, $_DECIMAL, $_ID }
%_COMPARACION_P = { ε, $_MAYOR, $_MENOR }
%_OP_REL = { $_MAYOR, $_MENOR }
%_LISTA_ARGS = { $_PAR_A, $_NUMERO, $_DECIMAL, $_ID }
%_LISTA_ARGS_P = { $_COMA, ε }

SIGUIENTE

%_PROGRAMA = { $ }
%_LISTA_SENT = { $_FIN, $_LLAVE_C }
%_LISTA_SENT_P = { $_FIN, $_LLAVE_C }
%_SENTENCIA = { $_ID, $_IF, $_WHILE, $_PRINT, $_RETURN, $_LLAVE_A, $_INT, $_FLOAT, $_FIN, $_LLAVE_C }
%_DECLARACION = { $_ID, $_IF, $_WHILE, $_PRINT, $_RETURN, $_LLAVE_A, $_INT, $_FLOAT, $_FIN, $_LLAVE_C }
%_TIPO = { $_ID }
%_ASIGNACION = { $_ID, $_IF, $_WHILE, $_PRINT, $_RETURN, $_LLAVE_A, $_INT, $_FLOAT, $_FIN, $_LLAVE_C }
%_SI_ENTONCES = { $_ID, $_IF, $_WHILE, $_PRINT, $_RETURN, $_LLAVE_A, $_INT, $_FLOAT, $_FIN, $_LLAVE_C }
%_SINO = { $_ID, $_IF, $_WHILE, $_PRINT, $_RETURN, $_LLAVE_A, $_INT, $_FLOAT, $_FIN, $_LLAVE_C }
%_MIENTRAS = { $_ID, $_IF, $_WHILE, $_PRINT, $_RETURN, $_LLAVE_A, $_INT, $_FLOAT, $_FIN, $_LLAVE_C }
%_IMPRIMIR = { $_ID, $_IF, $_WHILE, $_PRINT, $_RETURN, $_LLAVE_A, $_INT, $_FLOAT, $_FIN, $_LLAVE_C }
%_RETORNAR = { $_ID, $_IF, $_WHILE, $_PRINT, $_RETURN, $_LLAVE_A, $_INT, $_FLOAT, $_FIN, $_LLAVE_C }
%_BLOQUE = { $_ID, $_IF, $_WHILE, $_PRINT, $_RETURN, $_LLAVE_A, $_INT, $_FLOAT, $_FIN, $_ELSE, $_LLAVE_C }
%_EXPRESION = { $_PYC, $_MAYOR, $_MENOR, $_PAR_C, $_COMA }
%_EXPRESION_P = { $_PYC, $_MAYOR, $_MENOR, $_PAR_C, $_COMA }
%_TERMINO = { $_SUMA, $_RESTA, $_PYC, $_MAYOR, $_MENOR, $_PAR_C, $_COMA }
%_TERMINO_P = { $_SUMA, $_RESTA, $_PYC, $_MAYOR, $_MENOR, $_PAR_C, $_COMA }
%_FACTOR = { $_MULT, $_DIV, $_SUMA, $_RESTA, $_PYC, $_MAYOR, $_MENOR, $_PAR_C, $_COMA }
%_COMPARACION = { $_PAR_C }
%_COMPARACION_P = { $_PAR_C }
%_OP_REL = { $_PAR_A, $_NUMERO, $_DECIMAL, $_ID }
%_LISTA_ARGS = { $_PAR_C }
%_LISTA_ARGS_P = { $_PAR_C }

asi mismo tambien esta la opcion de mostrarle la tabla de analisis sintactico que es la que se utiliza para validar el analizador como tambien la tabla de simbolos

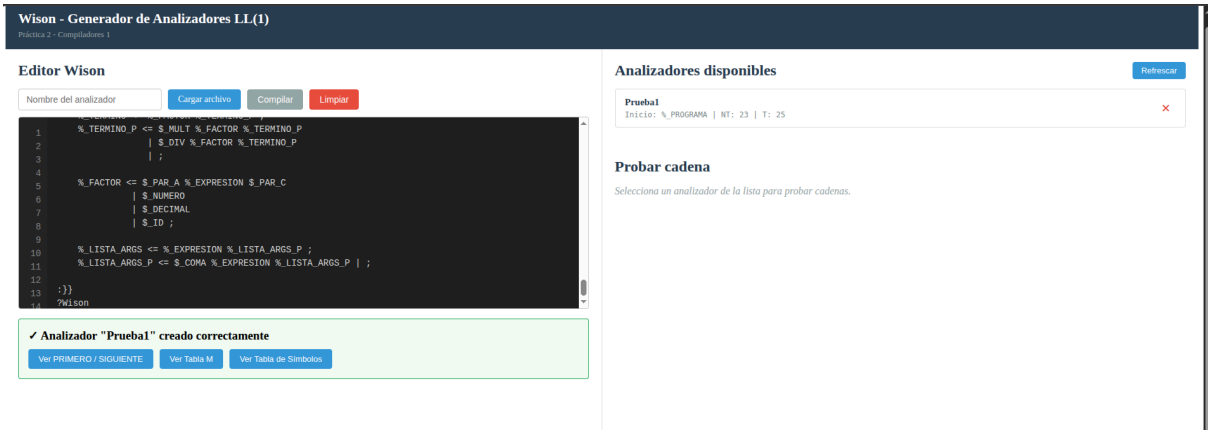
Tabla de Análisis Sintáctico (Tabla M)

	$_SUMA	$_RESTA	$_MULT	$_DIV	$_PAR_A	$_PAR_C	$_LLAVE_A	$_LLAVE_C	$_IGUAL	$_PYC	$_COMA	$_PUNTO	$_MAYOR
%_PROGRAMA							%_PROGRAMA → %_LISTA_SENT $_FIN						
%_LISTA_SENT							%_LISTA_SENT → %_SENTENCIA %_LISTA_SENT_P						
%_LISTA_SENT_P							%_LISTA_SENT_P → % SENTENCIA %_LISTA_SENT_P	%_LISTA_SENT_P → ε					
%_SENTENCIA							%_SENTENCIA → %_BLOQUE						
%_DECLARACION													
%_TIPO													
%_ASIGNACION													
%_SI_ENTONCES													
%_SINO							%_SINO → ε	%_SINO → ε					
%_MIENTRAS													

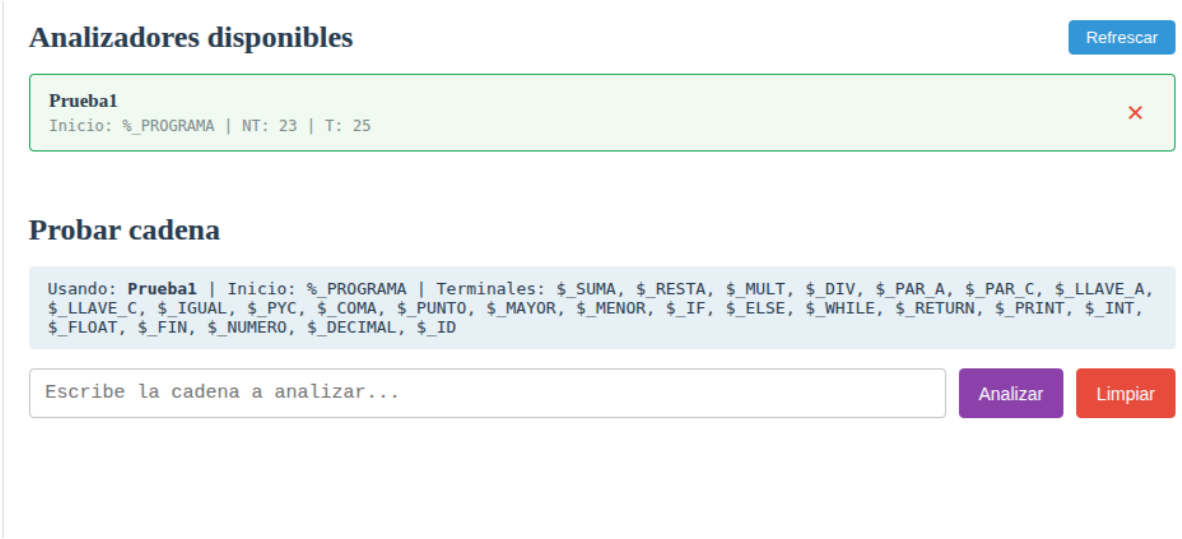
Tabla de Símbolos

$_MULT	Terminal	Léxico	Definido en bloque Lex
$_DIV	Terminal	Léxico	Definido en bloque Lex
$_PAR_A	Terminal	Léxico	Definido en bloque Lex
$_PAR_C	Terminal	Léxico	Definido en bloque Lex
$_LLAVE_A	Terminal	Léxico	Definido en bloque Lex
$_LLAVE_C	Terminal	Léxico	Definido en bloque Lex
$_IGUAL	Terminal	Léxico	Definido en bloque Lex
$_PYC	Terminal	Léxico	Definido en bloque Lex
$_COMA	Terminal	Léxico	Definido en bloque Lex
$_PUNTO	Terminal	Léxico	Definido en bloque Lex
$_MAYOR	Terminal	Léxico	Definido en bloque Lex
$_MENOR	Terminal	Léxico	Definido en bloque Lex
$_IF	Terminal	Léxico	Definido en bloque Lex
$_ELSE	Terminal	Léxico	Definido en bloque Lex
$_WHILE	Terminal	Léxico	Definido en bloque Lex
$_RETURN	Terminal	Léxico	Definido en bloque Lex
$_PRINT	Terminal	Léxico	Definido en bloque Lex
$_INT	Terminal	Léxico	Definido en bloque Lex
$_FLOAT	Terminal	Léxico	Definido en bloque Lex
$_FIN	Terminal	Léxico	Definido en bloque Lex
$_NUMERO	Terminal	Léxico	Definido en bloque Lex
$_DECIMAL	Terminal	Léxico	Definido en bloque Lex
$_ID	Terminal	Léxico	Definido en bloque Lex
%_PROGRAMA	No Terminal (Inicial)	Sintáctico	%_PROGRAMA → %_LISTA_SENT $_FIN
%_LISTA_SENT	No Terminal	Sintáctico	%_LISTA_SENT → %_SENTENCIA %_LISTA_SENT_P
%_LISTA_SENT_P	No Terminal	Sintáctico	%_LISTA_SENT_P → %_SENTENCIA %_LISTA_SENT_P | ε
%_SENTENCIA	No Terminal	Sintáctico	%_SENTENCIA → %_DECLARACION | %_ASIGNACION | %_SI_ENTONCES | %_MIENTRAS | %_IMPRIMIR | %_RETORNAR | %_BLOQUE
%_DECLARACION	No Terminal	Sintáctico	%_DECLARACION → %_TIPO $_ID $_PYC

3. Analisis de cadenas



cuando el usuario selecciona uno de los analizadores creados se le mostrara el espacio para ingresar la cadena a probar y mostrar tanto los resultados como el arbol de derivacion segun la cadena ingresada



cuando se ingrese la cadena se debe de presionar el boton de “Analizar” para que este ejecute el analisis usando el analizador seleccionado

si la cadena es valida entonces se mostrara el arbol de derivacion

Analizadores disponibles

Refrescar

Prueba1
Inicio: %_PROGRAMA | NT: 23 | T: 25

Probar cadena

Usando: **Prueba1** | Inicio: %_PROGRAMA | Terminales: $_SUMA, $_RESTA, $_MULT, $_DIV, $_PAR_A, $_PAR_C, $_LLAVE_A, $_LLAVE_C, $_IGUAL, $_PYC, $_COMA, $_PUNTO, $_MAYOR, $_MENOR, $_IF, $_ELSE, $_WHILE, $_RETURN, $_PRINT, $_INT, $_FLOAT, $_FIN, $_NUMERO, $_DECIMAL, $_ID

```
int x ; x = 1 + 2 ; if ( x > 5 ) { print ( x ) ; } else { return 0 ; } FIN
```

Analizar | Limpiar

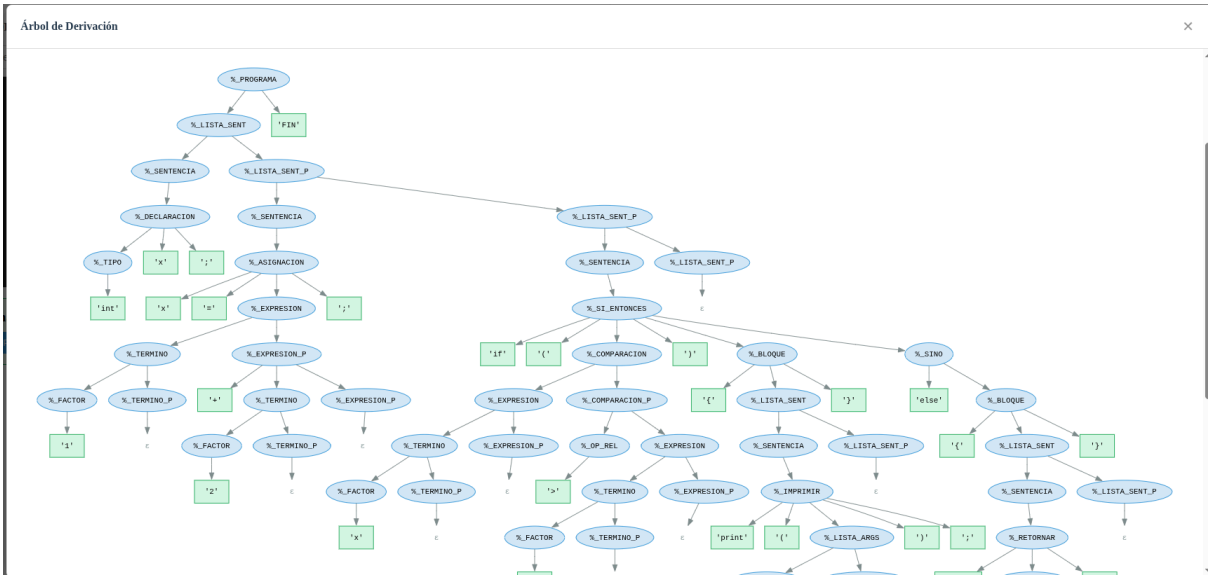
Tokens

$_INT: "int" | $_ID: "x" | $_PYC: ";" | $_ID: "x" | $_IGUAL: "=" | $_NUMERO: "1" | $_SUMA: "+" | $_NUMERO: "2" | $_PYC: ";" | $_IF: "if" | $_PAR_A: "(" | $_ID: "x" | $_MAYOR: ">" | $_NUMERO: "5" | $_PAR_C: ")" | $_LLAVE_A: "{" | $_PRINT: "print" | $_PAR_A: "(" | $_ID: "x" | $_PAR_C: ")" | $_PYC: ";" | $_LLAVE_C: "}" | $_ELSE: "else" | $_LLAVE_A: "{" | $_RETURN: "return" | $_NUMERO: "0" | $_PYC: ";" | $_LLAVE_C: "}" | $_FIN: "FIN"

✓ **Cadena aceptada**

Ver árbol de derivación

se debe de presionar en “ver arbol de derivacion” para ver como es que se desgloso la entrada



4. Errores

cuando se genera algun error la aplicacion es capaz de mostrar en donde fue el error tanto en la creacion de analizadores como en la entrada de cadenas para que el usuario pueda comprender en donde esta el fallo y que tipo de fallo es

analizador error | Cargar archivo | Compilar | Limpiar

```
86      No_Terminal %_LISTA_ARGS_P ;
87
88      # === Símbolo inicial ===
89      Initial_Sim %_PROGRAMA ;
90
91      # === Producciones ===
92
93      %_PROGRAMA <= %_LISTA_SENT $_FIN
94
95      %_LISTA_SENT <= %_SENTENCIA %_LISTA_SENT_P ;
96      %_LISTA_SENT_P <= %_SENTENCIA %_LISTA_SENT_P | ;
97
98      %_SENTENCIA <= %_DECLARACION
99                   | %_ASIGNACION
```

Errores encontrados (4)

LÍNEA	TIPO	DESCRIPCIÓN
-	LEXICO	Texto no reconocido "@adsfs" en linea 20
-	SINTACTICO	Se esperaba "<=" seguido de una produccion valida para "%_PROGRAMA" en linea 93.
-	SEMANTICO	No terminal "%_PROGRAMA" fue declarado pero no tiene ninguna producción definida.
-	SEMANTICO	No terminal "%_LISTA_SENT" fue declarado pero no tiene ninguna producción definida.

lo mismo pasa en el analisis de cadena, cuando una cadena no es valida se mostrar un mensaje indicando cual es el error

Probar cadena

Usando: **Prueba1** | Inicio: %_PROGRAMA | Terminales: $_SUMA, $_RESTA, $_MULT, $_DIV, $_PAR_A, $_PAR_C, $_LLAVE_A, $_LLAVE_C, $_IGUAL, $_PYC, $_COMA, $_PUNTO, $_MAYOR, $_MENOR, $_IF, $_ELSE, $_WHILE, $_RETURN, $_PRINT, $_INT, $_FLOAT, $_FIN, $_NUMERO, $_DECIMAL, $_ID

```
int x ; x = 1 + 2 ; if ( x > 5 ) { print ( x ) ; } else { return 0 ;
```

Analizar | Limpiar

Tokens

$_INT: "int" | $_ID: "x" | $_PYC: ";" | $_ID: "x" | $_IGUAL: "=" | $_NUMERO: "1" | $_SUMA: "+" | $_NUMERO: "2" | $_PYC: ";" | $_IF: "if" | $_PAR_A: "(" | $_ID: "x" | $_MAYOR: ">" | $_NUMERO: "5" | $_PAR_C: ")" | $_LLAVE_A: "{" | $_PRINT: "print" | $_PAR_A: "(" | $_ID: "x" | $_PAR_C: ")" | $_PYC: ";" | $_LLAVE_C: "}" | $_ELSE: "else" | $_LLAVE_A: "{" | $_RETURN: "return" | $_NUMERO: "0" | $_PYC: ";"

✕ **Cadena rechazada**

Ver árbol de derivación

Errores

[sintactico] Error en "%_LISTA_SENT_P": se encontró "$" pero se esperaba uno de: $_ID, $_IF, $_WHILE, $_PRINT, $_RETURN, $_LLAVE_A, $_INT, $_FLOAT, $_FIN, $_LLAVE_C.